

# UNIT - I

## Overview of NoSQL

*End Semester Examination Notes | SRM University Delhi-NCR | 23CAF3010*

Covers: Introduction to NoSQL | CAP Theorem | Data Models | Pros & Cons | SQL vs NoSQL | Document Databases

# 1. Introduction — Database & DBMS

---

A database is a software component that is critical in almost every modern application. A Database Management System (DBMS) is software used to store, organize, manage and retrieve data. It is a group of massive spreadsheets that organize information.

There are more than one type of database management system and each is housed on servers, whether in a data centre or virtually on cloud infrastructure (Cloud Database). Database management systems come in a variety of shapes, sizes and flavors, each designed to do different things with different kinds of data.

MongoDB is a general purpose, document-based, distributed database management system built for modern application developers.

## 1.1 Types of Database Management Systems

1. RDBMS (Relational Database Management System) — stores data in tables with rows and columns.
2. OLAP (Online Analytical Processing) — optimized for complex analytical queries.
3. NoSQL (Not only SQL) — non-relational databases with flexible schemas.

## 1.2 Information Stored in Database

Databases are used in most modern applications, whether on personal phones, computers or other Internet devices. An operational database system stores much of the data an application needs to function, keeping it organized and allowing users to access the data.

For example: Building an e-commerce app — data stored in the operational database includes:

- Customer data: usernames, email addresses, preferences
- Business data: product colours, prices and ratings
- Relationship data: locations of stores with a specific product in stock

## 1.3 Limitations of RDBMS (Why NoSQL was needed)

4. In relational databases, we need to define structure and schema of data first and then only we can process the data.
5. Relational database systems provide consistency and integrity of data by enforcing ACID properties (Atomicity, Consistency, Isolation and Durability). However in most other cases these properties are significant performance overhead and can make your database response very slow.
6. Most of the applications store their data in JSON format and RDBMS doesn't provide you a better way of performing operations such as Create, Insert, Update, Delete etc on this data.

★ **Exam Tip:** *ACID = Atomicity, Consistency, Isolation, Durability — remember this for 2-mark questions!*

## 2. NoSQL Database

---

Not only SQL (NoSQL) or non-relational database provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases. NoSQL databases don't use tables for storing data. It is generally used to store big data and real-time web applications.

NoSQL databases are non-relational data management systems built around flexible schemas, large data volume, low latency and specific data models. While relational database management systems such as MySQL and PostgreSQL organize and maintain structured data, they are resource-intensive and difficult to scale because they have to maintain the relationships as data is added.

NoSQL databases help solve these problems. Non-relational database management systems can scale vertically, similar to relational databases. However they can also scale horizontally — this means that NoSQL can add more machines or servers to the database to handle increased load.

### 2.1 Need of NoSQL Database

NoSQL databases were created in response to the limitations of traditional relational database technology. When compared against relational databases, NoSQL databases are more scalable and provide superior performance, and their data model addresses several shortcomings of the relational model.

The advantages of NoSQL include being able to handle:

7. Large volumes of structured, semi-structured and unstructured data.
8. Agile sprints, quick iterations and frequent code pushes.
9. Object-oriented programming that is easy to use and flexible.
10. Efficient, scale-out architecture instead of expensive, monolithic architecture.
11. Store and retrieve huge amounts of data.
12. The relationship between the data you store is not that important.
13. The data is not structured and changing over time.
14. Constraints and joins support is not required at a database level.
15. The data is growing continuously and you need to scale the database regularly to handle the data.

### 2.2 Four Kinds of NoSQL Database

16. Document Database — stores data in JSON, BSON or XML documents. Example: MongoDB, CouchDB.
17. Key-Value Stores — simplest type; every element stored as an attribute name (key) and a value.  
Example: Redis, DynamoDB.
18. Column-oriented Database — organized as a set of columns rather than rows. Example: Cassandra, HBase.
19. Graph Database — focuses on relationship between data elements; each element contained as a node.  
Example: Neo4J, RavenDB.

★ **Exam Tip:** *Four types of NoSQL databases is a very common 2-mark or 4-mark question. Must memorize all four with examples!*

## 3. Types of Databases (Detailed)

---

### 3.1 Hierarchical Databases

Developed in the 1960s, the hierarchical database looks similar to a family tree. A single object (the parent) has one or more objects beneath it (the child). No one can have more than one parent. In exchange for the rigid and complex navigation of the parent-child structure, the hierarchical database offers high performance, easy access and quick querying time.

### 3.2 Relational Databases

Relational databases are the system designed in 1970s. The database commonly uses Structured Query Language (SQL) for operations like creating, reading, updating and deleting data. This database stores data in discrete tables which can be joined together by fields known as foreign keys.

For example: we have a user table that contains data about users and join the user table to a purchase table which contains data about the purchases the users have made.

### 3.3 Non-Relational Databases (NoSQL)

Non-relational management systems are commonly referred to as NoSQL databases. This type of database matured due to increasingly complex modern web applications. These database varieties have proliferated over the last decade. Examples: MongoDB and Redis.

- Data can be stored in structured document similar to JSON — MongoDB is a popular document distribution database.
- Data can be in key-value format that maps a single attribute.
- A graph data store uses nodes to represent objects and edges to describe the relationship between them.

### 3.4 Object-Oriented Databases

Object-oriented databases store and manage objects on a database server disk. Object-oriented databases are unique because associations between objects can persist. This means that object-oriented programming and the querying of data across complex relationships is fast and powerful. Example: MongoDB Realm, where the query language constructs native objects through your chosen SDK.

### 3.5 How MongoDB Differs from Other Types of Databases

20. MongoDB is a NoSQL distributed database program. Because data doesn't need to fit within the confines of a strict relationship, MongoDB can operate as a general data store.
21. In this type of database, data is stored in MongoDB and maps to a flexible schema. If your application's needs change you can restructure how your data is stored.
22. Since MongoDB provides schema validation, you can lock down your schema as much or as little as you'd like. This means that the database can handle whatever your data structuring needs might be.
23. In relational databases, relationship between data in different tables can be achieved through joins and within hierarchical databases, relationships across nodes are impossible.
24. MongoDB provides the ability to join documents through operations that match a variety of needs such as \$lookup.

Lastly, MongoDB supports transactions, which guarantees the atomicity of reads and writes to multiple documents.

## 4. SQL vs NoSQL — Detailed Comparison

---

SQL (Structured Query Language) is a query language for storing, manipulating and retrieving data stored in a table structure (relational database). NoSQL (Not only SQL) provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases (non-relational database).

| SQL (Relational)                                                                                                                 | NoSQL (Non-Relational)                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Origin: Developed in the 1970s with a focus on reducing data duplication as storage was much more costly than developer time. | 1. Origin: Developed in the late 2000s with a focus on scaling, fast queries, allowing for frequent application changes and making programming simpler for developers. |
| 2. Database: SQL databases are primarily called RDBMS or relational databases.                                                   | 2. Database: NoSQL databases are primarily called non-relational or distributed databases.                                                                             |
| 3. Type: SQL databases are table-based databases.                                                                                | 3. Type: NoSQL databases can be document-based, key-value pairs or graph databases.                                                                                    |
| 4. Schema: Rigid (have fixed, static or predefined schemas).                                                                     | 4. Schema: Flexible (have dynamic schemas — schema can change on the fly).                                                                                             |
| 5. Relation: Data is having relations between tables.                                                                            | 5. Relation: No (or very few) relations between data.                                                                                                                  |
| 6. Distribution: Data is distributed across multiple tables.                                                                     | 6. Distribution: Data is typically merged/nested in a few collections.                                                                                                 |
| 7. Scaling: Vertically scalable (scale-up with a large server).                                                                  | 7. Scaling: Horizontally scalable (scale-out across commodity servers).                                                                                                |
| 8. Complex Queries: Best suited for complex queries.                                                                             | 8. Complex Queries: Not so good for complex queries.                                                                                                                   |
| 9. Performance: Requires specialized DB hardware for better performance.                                                         | 9. Performance: Uses commodity hardware.                                                                                                                               |
| 10. Examples: MySQL, Oracle, SQLite, PostgreSQL, MS-SQL etc.                                                                     | 10. Examples: MongoDB, BigTable, Redis, RavenDB, Cassandra, HBase, Neo4J, CouchDB etc.                                                                                 |

★ **Exam Tip:** *SQL vs NoSQL comparison table is almost guaranteed in the 16-mark section. Learn all 10 points with both columns!*

## 5. JSON vs BSON

---

| JSON                                          | BSON                                                                                     |
|-----------------------------------------------|------------------------------------------------------------------------------------------|
| Encoding: UTF-8 string (text-based format)    | Encoding: Binary (binary-encoded format)                                                 |
| Data Support: String, Boolean, Number, Array  | Data Support: String, Boolean, Number (Integer, Float, Decimal), Array, Date, Raw Binary |
| Readability: Human and Machine readable       | Readability: Machine only (binary cannot be read by humans directly)                     |
| Size: Larger (text takes more space)          | Size: Smaller (binary is more compact)                                                   |
| Speed: Slower to parse                        | Speed: Faster to parse and encode/decode                                                 |
| Usage: Used in API communication and frontend | Usage: Used internally by MongoDB for storage                                            |
| Extra types: Supports basic types only        | Extra types: Supports ObjectId, Date, Binary, Regex, Timestamp, MinKey, MaxKey           |

 **Note:** MongoDB accepts data in JSON format but stores it internally as BSON. At the backend, MongoDB converts JSON data into binary format known as BSON.

## 6. CAP Theorem (Brewer's Theorem)

---

The CAP theorem says that a distributed system can deliver only two of three desired characteristics: Consistency, Availability and Partition tolerance. It is also called Brewer's Theorem.

A distributed system is a network that stores data on more than one node (physical or virtual machines) at the same time. It is essential to understand the CAP theorem when designing a cloud app so that we choose a data management system that delivers the characteristics your applications needs the most.

### 6.1 Three Components of CAP Theorem

#### C — Consistency

Consistency means that all clients see the same data at the same time, no matter which node they connect to. For this to happen, whenever the data is written onto one node, it must be instantly forwarded or replicated to all the other nodes in the system before the write is deemed successful.

#### A — Availability

Availability means that any client making a request for data gets a response even if one or more nodes are down. Another way to state this — all working nodes in the distributed system return a valid response for any request, without exception.

#### P — Partition Tolerance

A Partition is a communication break within a distributed system — a lost or temporarily delayed connection between two nodes. Partition tolerance means that the cluster must continue to work despite any number of communication breakdowns between nodes in the system.

### 6.2 CAP Theorem — NoSQL Database Types

NoSQL databases are classified based on the 2 CAP characteristics they support:

| Database Type                                    | Description & Example                                                                                                                                                                                                                                                               |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CP Database (Consistency + Partition Tolerance)  | A CP database delivers consistency and partition tolerance at the expense of availability. When a partition occurs between any 2 nodes, the system has to shut down the non-consistent node until the partition is resolved. Example: MongoDB, HBase, BigTable.                     |
| AP Database (Availability + Partition Tolerance) | An AP database delivers availability and partition tolerance at the expense of consistency. When a partition occurs, all nodes remain available but those at the wrong end of a partition might return an older version of data than others. Example: Cassandra, CouchDB, DynamoDB. |
| CA Database (Consistency + Availability)         | A CA database delivers consistency and availability across all nodes. It can't do this if there is a partition between any two nodes in the system and therefore can't deliver fault tolerance. Example: MySQL, PostgreSQL (single node).                                           |

### 6.3 MongoDB and the CAP Theorem

MongoDB is a popular NoSQL database management system that stores data as BSON documents. It's frequently used for Big data and real-time applications running at multiple different locations.

Relative to the CAP theorem, MongoDB is a CP data store — it resolves network partitions by maintaining consistency, while compromising on availability.

- MongoDB is a single-master system — each replica set can select and have only one primary node that receives all the write operations.
- All other nodes in the same replica set are secondary nodes that replicate the primary node's operation log and apply it to their own data set.
- By default, clients can also read from the primary node but they can also specify a read preference that allows them to read from secondary nodes.
- When a primary node becomes unavailable, the secondary node with the most recent operation log will be elected as the new primary node.
- Once all the secondary nodes catch up with the new master, the cluster becomes available again.
- Clients can't make any write requests during this interval — that data remains consistent across the entire network.

## 6.4 Cassandra and the CAP Theorem

25. Apache Cassandra is an open-source NoSQL database maintained by the Apache Software Foundation.
26. It's a wide-column database that lets you store data on a distributed network.
27. However, unlike MongoDB, Cassandra has a masterless architecture and as a result it has multiple points of failure rather than single one.
28. Relative to the CAP theorem, Cassandra is an AP database — it delivers availability and partition tolerance but can't deliver consistency all the time.
29. Cassandra doesn't have a master node — all the nodes must be available continuously.
30. Cassandra provides eventual consistency by allowing clients to write to any nodes at any time and reconciling inconsistencies as quickly as possible.
31. As data only becomes inconsistent in the case of network partition, and inconsistencies are quickly resolved, Cassandra offers repair functionality to help nodes catch up with their peers.

## 6.5 Microservices and the CAP Theorem

32. Microservices are loosely coupled, independently deployable application components that incorporate their own stack — including their own database and database model — and communicate with each other over a network.
33. Understanding the CAP theorem can help choose the best database when designing a microservices-based application running from multiple locations.
34. For example: If the ability to quickly iterate the data model and scale horizontally is essential for an application, MongoDB (CP) would be a good choice.



## 7. Different Data Models in NoSQL

---

### 7.1 Document Database

A document database stores data in JSON, BSON or XML documents. Documents in the databases can be nested. Particular elements can be indexed for faster querying.

We can access, store and retrieve documents from your network in a form that is much closer to the data objects used in applications, which means less translation is required to use and access the data in an application.

SQL data must often be assembled and disassembled when moving between applications, storage or more than one network. Document databases are popular with developers because they offer the flexibility to rework their document structures as needed to suit their applications. This flexibility of this database speeds development — data becomes like a code and is under the control of developers.

Document databases are usually implemented with a scale-out architecture, providing a path to the scalability of data volumes and traffic.

### 7.2 Key-Value Stores

This is the simplest type of NoSQL database. Every element is stored as a key-value pair consisting of an attribute name (key) and a value. This database is like an RDBMS with 2 columns: the attribute name (such as state) and the value (such as Alaska). Example: Redis.

### 7.3 Column-Oriented Database

35. While an RDBMS stores data in rows and reads it row by row, column-oriented databases are organized as a set of columns.
36. When we want to run analytics on a small number of columns in the network, we can read those columns directly without consuming data with unwanted data.
37. Columns are of the same type and benefit from more efficient comparison, making reads even faster.
38. A column-oriented database can aggregate the value of a given column.
39. While column-oriented databases are great for analytics, the way they write data makes it difficult for them to be consistent as writes of all the columns in the column-oriented database require multiple write events on disk.
40. Relational databases don't suffer from this problem as row data is written contiguously to disk.

### 7.4 Graph Databases

41. A graph database focuses on the relationship between data elements. Each element is stored/contained as a node.
42. The connections between elements in the database are called links or relationships.
43. A graph database is optimized to capture and search connections between elements.
44. These databases are diverse in their data formats and applications.

## 8. Pros & Cons of Using NoSQL

---

### 8.1 Pros of NoSQL (10 Advantages)

45. Performance: NoSQL databases can often outperform SQL databases simply because all the info is contained inside one database. No need for expensive joins.
46. Scalability: The second biggest positive of NoSQL is that it is horizontally rather than vertically scalable. Adding more servers is much cheaper than upgrading one big server.
47. Flexibility: Due to the non-rigid nature of NoSQL, it is much easier to test ideas and update. No schema migration required.
48. Data Models: NoSQL is more of a philosophy than a specific data model and has several data models beneath it. The data models tend to be incredibly specialized in specific use cases, allowing them to outperform relational databases.
49. Large Amounts of Data Storage: Many NoSQL databases can handle extensive datasets making them ideal for big data applications and IoT.
50. Simplicity: Many NoSQL database management systems require only a few lines of code, which is ideal for developers who want to get started quickly.
51. Less Maintenance: NoSQL databases don't require the same level of ongoing database administration as SQL databases.
52. Continuous Availability: With NoSQL, data is distributed across multiple servers and regions, so there is no single point of failure.
53. Query Speed: Since NoSQL databases are de-normalized with no worry of data duplication, all the info needed for a particular query will often already be stored together.
54. Low Cost: NoSQL databases scale-out horizontally making it cost-effective to expand capacity.

### 8.2 Cons of NoSQL (9 Disadvantages)

55. Not Mature — NoSQL is relatively newer technology and is not as mature as relational databases.
56. Requires multiple databases — Different NoSQL databases may be needed for different use cases.
57. Huge databases — Can result in very large databases due to data duplication.
58. No standardized language — Unlike SQL which is standardized, each NoSQL database has its own query language.
59. Smaller user community — Less community support compared to relational databases.
60. Inefficiency with complex queries — NoSQL is not optimized for complex JOIN-type queries.
61. Data retrieval inconsistency — Due to eventual consistency model, data may be inconsistent during retrieval.
62. Queries are less flexible — Querying in NoSQL can be less flexible than SQL's powerful query language.
63. NoSQL isn't designed to scale by itself — Horizontal scaling requires proper configuration.

### 8.3 Pros of RDBMS (for comparison)

- They're ubiquitous, having acquired a steady user base since the 1970s.
- They're optimized for working with structured data.
- They provide a stronger guarantee of data normalization.
- They use a well-known querying language (SQL).

## 9. Characteristics of NoSQL Databases

---

### 9.1 Schema-less Design

NoSQL databases typically do not enforce a fixed schema. This allows developers to insert data without first defining a structure, making them more adaptable to evolving data models.

### 9.2 Variety of Data Models

NoSQL databases support various data models, including document-oriented, key-value pairs, wide-column stores and graph databases. This versatility allows developers to choose the best-fit data model for their specific application needs.

### 9.3 Horizontal Scalability

NoSQL databases are designed to scale horizontally — meaning they can handle increased data volumes and traffic by adding more servers or nodes to their database system. This is in contrast to SQL which scales vertically (adding more power to one server).

### 9.4 Distributed Architecture

Many NoSQL databases are built to operate in distributed environments, allowing them to distribute data across multiple nodes or servers. This design enhances fault tolerance and ensures data availability.

### 9.5 Dynamic and Unstructured Data

NoSQL databases are allowed and are well suited for managing unstructured or semi-structured data such as JSON or XML documents. This makes them versatile in handling diverse data formats.

## 10. Basic MongoDB Commands (from Notes)

---

### 10.1 Database Commands

Create Database:

```
use database_name
// If the database doesn't exist, above command creates a new database,
// otherwise opens the existing one.
```

Delete Database:

```
db.dropDatabase()
```

List all Databases:

```
show dbs
```

### 10.2 Collection Commands

MongoDB stores documents in collections. Collections are analogous to tables in relational databases. A collection exists within a single database. Collections do not enforce a schema. A collection may store documents those which are not same in structure because it is a schema-free database.

Method 1 — Implicit Creation (with insert):

```
// In MongoDB you need not to create collection before you insert document in it.
// With a single command you can insert a document in the collection and
// MongoDB creates that collection on the fly.
db.collection_name.insert({ key: value, key: value })
```

Method 2 — Explicit Creation (createCollection):

```
db.createCollection(name, options)
// name → collection name
// options → optional field that we can use to specify certain
//           parameters such as size, max number of documents
```

Delete a Collection:

```
// To drop a collection, first connect to the database in which
// you want to delete the collection.
db.collection_name.drop()
```

### 10.3 Document / CRUD Commands

MongoDB stores data records as BSON documents. BSON is a binary representation of JSON documents.

```
{
  field1: value1,
  field2: value2,
  field3: value3,
  ...
  fieldN: valueN
}
```

Query: A Query is a request for data or information from a database table or combination of tables or collections (in case of NoSQL database). This data may be generated as results returned by query or as pictorials, graphs or complex results such as trend analyses from data mining tools.

CRUD Operations:

```
// CREATE (Insert)
db.collection.insertOne({ key: value, key: value })
db.collection.insertMany([ { key: value }, { key: value } ])

// READ (Find)
```

```

db.collection.find(query, projection)

// UPDATE
db.collection_name.updateOne(<filter>, <update>)
db.collection_name.updateMany(<filter>, <update>)

// DELETE
db.collection_name.deleteOne(delete_criteria)
db.collection_name.deleteMany(deletion_criteria)

```

## 10.4 Query Operators with Examples

We can add criteria to our queries so that we can fetch documents based on certain conditions. Instead of fetching all documents from a collection, we can fetch selected documents based on criteria.

| Operation           | Syntax                        | Example                                                                                                    |
|---------------------|-------------------------------|------------------------------------------------------------------------------------------------------------|
| Equality            | { <key>: <value> }            | db.Employee.find({ name: 'Aman' })                                                                         |
| Less Than           | { <key>: { \$lt: <value> } }  | db.Employee.find({ basicSalary: { \$lt: 12000 } })                                                         |
| Less Than Equals    | { <key>: { \$lte: <value> } } | db.Employee.find({ basicSalary: { \$lte: 12000 } })                                                        |
| Greater Than        | { <key>: { \$gt: <value> } }  | db.Employee.find({ age: { \$gt: 25 } })                                                                    |
| Greater Than Equals | { <key>: { \$gte: <value> } } | db.Employee.find({ basicSalary: { \$gte: 12000 } })                                                        |
| Not Equals          | { <key>: { \$ne: <value> } }  | db.Employee.find({ city: { \$ne: 'Bangalore' } })                                                          |
| AND                 | { \$and: [{cond1}, {cond2}] } | db.Employee.find({ \$and: [{ basicSalary: 20000 }, { city: 'Bangalore' }] })                               |
| OR                  | { \$or: [{cond1}, {cond2}] }  | db.Employee.find({ \$or: [{ basicSalary: 20000 }, { city: 'Bangalore' }] })                                |
| AND + OR            | Combine both                  | db.Employee.find({ basicSalary: { \$gte: 12000 }, \$or: [{ city: 'Bangalore' }, { age: { \$gte: 22 } }] }) |

# 11. Early Database Systems

---

| Database                      | Advantages                                                                | Limitations                                                       |
|-------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------|
| 1. Flat File System           | Simplest form of database storing data in plain text files.               | Limited in handling complex relationships between data.           |
| 2. Hierarchical Systems       | Organized data in a tree-like structure with parent-child relationship.   | Changes to the structure were challenging to implement.           |
| 3. Network Database System    | Modeled data using a graph structure allowing many-to-many relationships. | Complexity in designing and maintaining the database schemas.     |
| 4. Relational Database System | Organized data into tables with rows and columns. Easy to query.          | Faces scalability challenges in big data with high traffic loads. |

## EXAMINATION PREPARATION — UNIT 1

### PART A: 2-Mark Questions with Answers (10 Questions)

#### Q1. What is NoSQL? When was it developed?

NoSQL (Not only SQL) or non-relational database provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases. NoSQL databases don't use tables for storing data and are generally used to store big data and real-time web applications. NoSQL was developed in the late 2000s with a focus on scaling, fast queries, allowing for frequent application changes and making programming simpler for developers.

---

#### Q2. What is the CAP Theorem? Name its three components.

The CAP theorem (Brewer's Theorem) states that a distributed system can deliver only two of three desired characteristics. The three components are:

- 64. Consistency (C) — All clients see the same data at the same time, no matter which node they connect to.
  - 65. Availability (A) — Any client making a request for data gets a response even if one or more nodes are down.
  - 66. Partition Tolerance (P) — The cluster must continue to work despite any number of communication breakdowns between nodes.
- 

#### Q3. What are the four types of NoSQL databases? Give one example each.

- 67. Document Database — stores data in JSON/BSON/XML. Example: MongoDB, CouchDB.
  - 68. Key-Value Stores — simplest type, stores key-value pairs. Example: Redis, DynamoDB.
  - 69. Column-Oriented Database — organized as columns not rows. Example: Cassandra, HBase.
  - 70. Graph Database — focuses on relationships between data. Example: Neo4J, RavenDB.
- 

#### Q4. What is the difference between CP and AP databases in CAP theorem?

CP Database (Consistency + Partition Tolerance): Delivers consistency and partition tolerance at the expense of availability. When a partition occurs, the system shuts down the non-consistent node until the partition is resolved. Example: MongoDB. AP Database (Availability + Partition Tolerance): Delivers availability and partition tolerance at the expense of consistency. When a partition occurs, all nodes remain available but may return an older version of data. Example: Cassandra.

---

#### Q5. What is ACID? Why does it cause problems in RDBMS?

ACID stands for Atomicity, Consistency, Isolation and Durability. Relational database systems provide consistency and integrity of data by enforcing ACID properties. However, in most cases these properties are significant performance overhead and can make the database response very slow. This is one of the major limitations of RDBMS, especially for large-scale, real-time applications. NoSQL relaxes ACID properties to gain better performance and scalability.

---

#### Q6. What is the difference between JSON and BSON?

JSON (JavaScript Object Notation) is a text-based format encoded in UTF-8 that supports String, Boolean, Number and Array types. It is human and machine readable. BSON (Binary JSON) is a binary-encoded format that supports additional data types including Integer, Float, Decimal, Date, Raw Binary, ObjectId etc. BSON is machine-only readable, faster to parse and more compact. MongoDB accepts data in JSON format but stores it internally as BSON.

---

### **Q7. What is horizontal scaling vs vertical scaling?**

Vertical Scaling (Scale-Up): Adding more power (CPU, RAM, storage) to an existing server. Used by SQL/RDBMS databases. Has a physical limit. Expensive. Horizontal Scaling (Scale-Out): Adding more servers/nodes to the system. Used by NoSQL databases. No practical limit. Cost-effective using commodity hardware. NoSQL can add more machines or servers to the database to handle increased load, while SQL databases are limited to upgrading a single large server.

---

### **Q8. What is a Document Database? Give two examples.**

A document database stores data in JSON, BSON or XML documents. Documents in the databases can be nested and particular elements can be indexed for faster querying. We can access, store and retrieve documents from a network in a form that is much closer to data objects used in applications, which means less translation is required. Document databases are popular with developers because they offer flexibility to rework document structures as needed. Examples: MongoDB and CouchDB.

---

### **Q9. List any five advantages (Pros) of NoSQL databases.**

71. Performance: Can outperform SQL as all info is contained in one database — no expensive joins.
  72. Scalability: Horizontally scalable — add commodity servers to handle more load.
  73. Flexibility: Non-rigid nature makes it easy to test ideas and update without schema migration.
  74. Continuous Availability: Data distributed across multiple servers — no single point of failure.
  75. Low Cost: Scale-out horizontally making it cost-effective to expand capacity.
- 

### **Q10. What are the three basic commands to manage databases in MongoDB?**

Three basic MongoDB database commands:

76. Create Database: use database\_name — creates a new database if it doesn't exist, otherwise opens the existing one.
77. Delete Database: db.dropDatabase() — removes the specified database and associated data files.
78. List all Databases: show dbs — displays all databases on the MongoDB server.



## **16M-Q1. Explain NoSQL database in detail. What is the need for NoSQL? Explain its advantages and disadvantages. (16 Marks)**

### **Introduction — What is NoSQL (2 marks):**

NoSQL (Not only SQL) or non-relational database provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases. NoSQL databases don't use tables for storing data and are generally used to store big data and real-time web applications. NoSQL databases are non-relational data management systems built around flexible schemas, large data volume, low latency and specific data models.

### **Need for NoSQL — Limitations of RDBMS (3 marks):**

NoSQL databases were created in response to the limitations of traditional relational database technology:

79. In relational databases, we need to define structure and schema of data first and then only we can process the data — rigid structure.
80. RDBMS enforces ACID properties which are significant performance overhead and can make the database response very slow.
81. Most applications store data in JSON format and RDBMS doesn't provide efficient operations (Create, Insert, Update, Delete) on this data.
82. RDBMS can only scale vertically — adding more CPU/RAM to one server — which has physical and cost limits.

NoSQL is needed when:

- Large volumes of structured, semi-structured and unstructured data need to be handled.
- The data is not structured and is changing over time.
- The data is growing continuously and needs horizontal scaling.
- Constraints and joins support is not required at database level.

### **Advantages of NoSQL — Pros (6 marks):**

83. Performance: Can outperform SQL databases as all the info is contained inside one database — no expensive joins needed.
84. Scalability: Horizontally scalable (scale-out across commodity servers) rather than vertically scalable. Can add more machines to handle load.
85. Flexibility: Due to the non-rigid nature of NoSQL, it is much easier to test ideas and update. No schema migration required.
86. Large Data Storage: Many NoSQL databases can handle extensive datasets — ideal for big data applications and IoT.
87. Continuous Availability: Data distributed across multiple servers and regions — no single point of failure.
88. Query Speed: De-normalized databases — all info for a particular query often already stored together.
89. Low Cost: Scale-out horizontally making it cost-effective to expand capacity.
90. Simplicity: Many NoSQL database management systems require only a few lines of code.
91. Less Maintenance: Don't require the same level of ongoing database administration as SQL databases.
92. Data Models: Several specialized data models tend to outperform relational databases in specific use cases.

### **Disadvantages of NoSQL — Cons (3 marks):**

93. Not Mature: NoSQL is relatively newer technology and is not as mature as relational databases.

- 94. No Standardized Language: Unlike SQL which is standardized, each NoSQL database has its own query language.
- 95. Smaller User Community: Less community support compared to relational databases.
- 96. Inefficiency with Complex Queries: Not optimized for complex JOIN-type queries.
- 97. Data Retrieval Inconsistency: Due to eventual consistency model, data may be inconsistent during retrieval.
- 98. NoSQL isn't designed to scale by itself — horizontal scaling requires proper configuration.

### **Conclusion (2 marks):**

NoSQL databases offer superior performance, scalability and flexibility for modern big data and real-time applications. While they have some limitations like lack of standardization and maturity, their ability to handle unstructured data at scale makes them indispensable for modern application development. MongoDB is the most popular NoSQL database, combining the best of document storage with rich querying capabilities.

---

## **16M-Q2. Explain the CAP Theorem in detail with its three components and types of NoSQL databases based on CAP. Explain MongoDB's and Cassandra's position in CAP. (16 Marks)**

### **Introduction — CAP Theorem (2 marks):**

The CAP theorem (also called Brewer's Theorem) states that a distributed system can deliver only two of three desired characteristics: Consistency, Availability and Partition tolerance. A distributed system is a network that stores data on more than one node (physical or virtual machines) at the same time. It is essential to understand the CAP theorem when designing a cloud app so that we choose a data management system that delivers the characteristics our application needs the most.

### **Three Components of CAP Theorem (6 marks — 2 each):**

#### **C — Consistency (2 marks):**

Consistency means that all clients see the same data at the same time, no matter which node they connect to. For this to happen, whenever the data is written onto one node, it must be instantly forwarded or replicated to all the other nodes in the system before the write is deemed successful.

#### **A — Availability (2 marks):**

Availability means that any client making a request for data gets a response even if one or more nodes are down. Another way to state this: all working nodes in the distributed system return a valid response for any request, without exception.

#### **P — Partition Tolerance (2 marks):**

A Partition is a communication break within a distributed system — a lost or temporarily delayed connection between two nodes. Partition tolerance means that the cluster must continue to work despite any number of communication breakdowns between nodes in the system.

### **CAP Theorem — Database Types (4 marks):**

#### **CP Database (Consistency + Partition Tolerance):**

Delivers consistency and partition tolerance at the expense of availability. When a partition occurs between any 2 nodes, the system has to shut down the non-consistent node until the partition is resolved. Example: MongoDB, HBase, BigTable.

#### **AP Database (Availability + Partition Tolerance):**

Delivers availability and partition tolerance at the expense of consistency. When a partition occurs, all nodes remain available but those at the wrong end of a partition might return an older version of data than others. Example: Cassandra, CouchDB, DynamoDB.

#### **CA Database (Consistency + Availability):**

Delivers consistency and availability across all nodes. It can't do this if there is a partition between any two nodes in the system. Example: MySQL, PostgreSQL (single node).

### **MongoDB and CAP Theorem (2 marks):**

MongoDB is a CP data store — it resolves network partitions by maintaining consistency, while compromising on availability. MongoDB is a single-master system — each replica set can have only one primary node that receives all write operations. All other nodes are secondary nodes. When a primary node fails, the secondary with the most recent operation log is elected as the new primary automatically. Clients cannot make write requests during this interval, ensuring data remains consistent.

### **Cassandra and CAP Theorem (2 marks):**

Cassandra is an AP database — it delivers availability and partition tolerance but can't deliver consistency all the time. Cassandra has masterless architecture with multiple points of failure rather than single one. Cassandra

provides eventual consistency by allowing clients to write to any node at any time and reconciling inconsistencies as quickly as possible.

---

## 16M-Q3. Compare SQL and NoSQL databases in detail with at least 10 points. Explain different types of NoSQL databases with examples. (16 Marks)

### Introduction (1 mark):

SQL (Structured Query Language) is a query language for storing, manipulating and retrieving data stored in a table structure (relational database). NoSQL (Not only SQL) provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases.

### SQL vs NoSQL Comparison — 10 Points (10 marks):

99. Origin: SQL developed in 1970s to reduce data duplication. NoSQL developed in late 2000s for scaling, fast queries and frequent application changes.
100. Database Name: SQL databases called RDBMS or relational databases. NoSQL called non-relational or distributed databases.
101. Type: SQL is table-based. NoSQL can be document-based, key-value pairs or graph databases.
102. Schema: SQL is rigid (fixed, static or predefined schemas). NoSQL is flexible (dynamic schemas, can change on the fly).
103. Relation: SQL data has relations between tables. NoSQL has no (or very few) relations.
104. Distribution: SQL data distributed across multiple tables. NoSQL data typically merged/nested in a few collections.
105. Scaling: SQL is vertically scalable (scale-up with a large server). NoSQL is horizontally scalable (scale-out across commodity servers).
106. Complex Queries: SQL best suited for complex queries. NoSQL not so good for complex queries.
107. Performance: SQL requires specialized DB hardware. NoSQL uses commodity hardware.
108. Examples: SQL — MySQL, Oracle, SQLite, PostgreSQL, MS-SQL. NoSQL — MongoDB, Redis, Cassandra, HBase, Neo4J, CouchDB.

### Types of NoSQL Databases (5 marks):

#### 1. Document Database (2 marks):

Stores data in JSON, BSON or XML documents. Documents can be nested. Particular elements can be indexed for faster querying. Popular with developers for flexibility. Implemented with scale-out architecture. Examples: MongoDB, CouchDB.

#### 2. Key-Value Stores (1 mark):

Simplest type of NoSQL database. Every element stored as a key-value pair consisting of an attribute name (key) and a value. Like an RDBMS with 2 columns: attribute name and value. Example: Redis, DynamoDB.

#### 3. Column-Oriented Database (1 mark):

While RDBMS stores data in rows, column-oriented databases are organized as a set of columns. Best for analytics on small number of columns. Columns are of the same type enabling efficient comparison. Example: Cassandra, HBase.

#### 4. Graph Database (1 mark):

Focuses on relationships between data elements. Each element stored as a node. Connections between elements called links or relationships. Optimized to capture and search connections. Example: Neo4J, RavenDB.

---

## 16M-Q4. Explain Document Databases in detail. What are the different types of databases? Explain with their early database systems. (16 Marks)

### Introduction to Document Databases (4 marks):

A document database stores data in JSON, BSON or XML documents. Documents in the databases can be nested and particular elements can be indexed for faster querying. We can access, store and retrieve documents from a network in a form that is much closer to the data objects used in applications — which means less translation is required to use and access the data in an application.

SQL data must often be assembled and disassembled when moving between applications, storage or more than one network. Document databases are popular with developers because they offer the flexibility to rework their document structures as needed to suit their applications. This flexibility speeds development — data becomes like code and is under the control of developers.

Document databases are usually implemented with a scale-out architecture, providing a path to the scalability of data volumes and traffic.

### Types of Databases — Full Classification (8 marks):

#### A. Hierarchical Databases (2 marks):

Developed in the 1960s, looks similar to a family tree. A single object (parent) has one or more objects beneath it (child). No one can have more than one parent. Offers high performance, easy access and quick querying time in exchange for rigid parent-child structure.

#### B. Relational Databases (2 marks):

Designed in 1970s. Commonly uses SQL for creating, reading, updating and deleting data. Stores data in discrete tables that can be joined together by foreign keys. Example: we have a user table joined to a purchase table using a foreign key.

#### C. Non-Relational Databases (NoSQL) (2 marks):

Commonly referred to as NoSQL databases. Matured due to increasingly complex modern web applications. Data can be stored in structured document (JSON), key-value format or graph data store (nodes and edges). Examples: MongoDB, Redis.

#### D. Object-Oriented Databases (2 marks):

Store and manage objects on a database server disk. Unique because associations between objects can persist. Object-oriented programming and querying of data across complex relationships is fast and powerful. Example: MongoDB Realm.

### Early Database Systems (4 marks):

| Database                   | Advantage                                                                 | Limitation                                                        |
|----------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------|
| Flat File System           | Simplest form, stores data in plain text files.                           | Limited in handling complex relationships between data.           |
| Hierarchical System        | Organized data in tree-like structure with parent-child relationship.     | Changes to structure were challenging to implement.               |
| Network Database System    | Modeled data using graph structure, allowing many-to-many relationships.  | Complexity in designing and maintaining database schemas.         |
| Relational Database System | Organized data into tables with rows and columns, easy to query with SQL. | Faces scalability challenges in big data with high traffic loads. |

## 16M-Q5. Explain the Characteristics of NoSQL databases. Explain CRUD operations in MongoDB with syntax and examples. (16 Marks)

### 1. Characteristics of NoSQL Databases (8 marks):

#### 1. Schema-less Design (2 marks):

NoSQL databases typically do not enforce a fixed schema. This allows developers to insert data without first defining a structure, making them more adaptable to evolving data models. Unlike RDBMS where schema must be defined before data insertion, NoSQL allows data to be inserted in any structure at any time.

#### 2. Variety of Data Models (2 marks):

NoSQL databases support various data models including document-oriented (MongoDB), key-value pairs (Redis), wide-column stores (Cassandra) and graph databases (Neo4J). This versatility allows developers to choose the best-fit data model for their specific application needs.

#### 3. Horizontal Scalability (2 marks):

NoSQL databases are designed to scale horizontally — meaning they can handle increased data volumes and traffic by adding more servers or nodes to their database system. This contrasts with SQL's vertical scaling (adding more power to one server). Horizontal scaling is more cost-effective using commodity hardware.

#### 4. Distributed Architecture (1 mark):

Many NoSQL databases are built to operate in distributed environments, allowing them to distribute data across multiple nodes or servers. This design enhances fault tolerance and ensures data availability.

#### 5. Dynamic and Unstructured Data (1 mark):

NoSQL databases are well suited for managing unstructured or semi-structured data such as JSON or XML documents. This makes them versatile in handling diverse data formats like social media data, sensor data, log files etc.

### 2. CRUD Operations in MongoDB (8 marks):

CRUD stands for Create, Read, Update and Delete — the four fundamental operations of any database.

#### CREATE — Insert Operations (2 marks):

```
// Insert one document
db.collection.insertOne({ name: 'Alice', age: 25, city: 'Delhi' })

// Insert multiple documents
db.collection.insertMany([
  { name: 'Bob', age: 30 },
  { name: 'Carol', age: 22 }
])
```

#### READ — Find Operations (2 marks):

```
// Find all documents
db.collection.find({})

// Find with condition (age > 25)
db.Employee.find({ age: { $gt: 25 } }).pretty()

// Find with AND
db.Employee.find({ $and: [{ basicSalary: 20000 }, { city: 'Bangalore' }] })

// Find with OR
db.Employee.find({ $or: [{ basicSalary: 20000 }, { city: 'Bangalore' }] })
```

#### UPDATE — Update Operations (2 marks):

```
// Update one document
db.collection_name.updateOne(<filter>, <update>)
```

```
// Example:
db.students.updateOne(
  { name: 'Alice' },
  { $set: { age: 26, status: 'Active' } }
)

// Update many documents
db.students.updateMany(
  { city: 'Delhi' },
  { $set: { status: 'Verified' } }
)
```

### **DELETE — Delete Operations (2 marks):**

```
// Delete one document
db.collection_name.deleteOne(delete_criteria)
db.students.deleteOne({ name: 'Bob' })

// Delete many documents
db.collection_name.deleteMany(deletion_criteria)
db.students.deleteMany({ city: 'Delhi' })

// Delete all documents (empty filter)
db.students.deleteMany({})
```



## QUICK REVISION — UNIT 1 KEY FACTS

| Topic                      | Key Fact to Remember                                                                                               |
|----------------------------|--------------------------------------------------------------------------------------------------------------------|
| Full form of NoSQL         | Not Only SQL (non-relational database)                                                                             |
| NoSQL developed in         | Late 2000s (for scaling, fast queries, flexibility)                                                                |
| SQL developed in           | 1970s (for reducing data duplication)                                                                              |
| ACID full form             | Atomicity, Consistency, Isolation, Durability                                                                      |
| CAP Theorem full form      | Consistency, Availability, Partition Tolerance                                                                     |
| CAP theorem also called    | Brewer's Theorem                                                                                                   |
| MongoDB in CAP             | CP database (Consistency + Partition Tolerance)                                                                    |
| Cassandra in CAP           | AP database (Availability + Partition Tolerance)                                                                   |
| CP Database trade-off      | Sacrifices Availability for Consistency                                                                            |
| AP Database trade-off      | Sacrifices Consistency for Availability                                                                            |
| 4 types of NoSQL           | Document, Key-Value, Column-oriented, Graph                                                                        |
| Document DB example        | MongoDB, CouchDB                                                                                                   |
| Key-Value DB example       | Redis, DynamoDB                                                                                                    |
| Column-oriented DB example | Cassandra, HBase                                                                                                   |
| Graph DB example           | Neo4J, RavenDB                                                                                                     |
| SQL scaling                | Vertical (scale-up — add power to one server)                                                                      |
| NoSQL scaling              | Horizontal (scale-out — add more servers)                                                                          |
| JSON encoding              | UTF-8 string (human + machine readable)                                                                            |
| BSON encoding              | Binary (machine only, faster, more types)                                                                          |
| MongoDB stores as          | BSON documents internally (accepts JSON)                                                                           |
| SQL examples               | MySQL, Oracle, SQLite, PostgreSQL, MS-SQL                                                                          |
| NoSQL examples             | MongoDB, Redis, Cassandra, HBase, Neo4J, CouchDB                                                                   |
| Schema in SQL              | Rigid / Fixed / Predefined                                                                                         |
| Schema in NoSQL            | Flexible / Dynamic (can change on fly)                                                                             |
| 5 Characteristics of NoSQL | Schema-less, Variety of data models, Horizontal scalability, Distributed architecture, Dynamic & unstructured data |
| NoSQL Pros (key ones)      | Performance, Scalability, Flexibility, Low Cost, Continuous Availability                                           |
| NoSQL Cons (key ones)      | Not Mature, No Standardized Language, Smaller community, Inefficient complex queries                               |
| CRUD stands for            | Create, Read, Update, Delete                                                                                       |

|                      |                           |
|----------------------|---------------------------|
| Create DB in MongoDB | use database_name         |
| Delete DB in MongoDB | db.dropDatabase()         |
| List all DBs         | show dbs                  |
| Drop a Collection    | db.collection_name.drop() |

**All the Best for Your End Semester Examinations!**

*Unit 1 — Overview of NoSQL | SRM University Delhi-NCR | Subject Code: 23CAF3010*